Dynamic Context-Aware Tries For Knowledge Graph-Constained Large Language Model Generation

Bernard Kirk Adjanor Katamanso
Erica Amanor
Joseph Osei Nyarko
Jessica Afua Etornam Nsafoah

Supervised By: Dr. Eric Affum

University of Mines and Technology
Faculty of Computing and Mathematical Science
Computer Science and Engineering Department

July ,2026

Outline

Introduction

- LLMs hallucinate, producing plausible but factually incorrect outputs (Brown et al., 2020; Ji et al., 2023).
- KGQA requires answers grounded in real graph structure (He et al., 2021; Lan et al., 2023).
- Constrained decoding methods (GCR, DoG) guarantee structural validity but not semantic relevance (Geng et al., 2023; Li et al., 2025; Luo et al., 2025).
- A 3-hop question on Freebase can produce ∼24,000 structurally valid paths, yet only one leads to the correct answer.

Statement of Problem

- GCR's trie is fixed before decoding starts, so it cannot adapt to question intent or the evolving reasoning chain (Luo et al., 2025).
- This forces the LLM to filter semantically irrelevant paths using internal knowledge, the same mechanism that causes hallucinations (Li et al., 2025).
- Existing work assumes tighter constraints improve accuracy, but no metric exists to measure constraint quality independently of answer quality.

Project Objectives (1/2)

- Characterize the permissiveness problem by measuring SIR of GCR's static KG-Tries on WebQSP
- Design a symbolic constraint oracle (TypeOracle) with range and type gates, avoiding embedding cost and threshold sensitivity

Project Objectives (2/2)

- Implement DCA-Trie v1 (static filtering at construction time, FNR $<$ 5%) and v2 (dynamic expansion per committed triplet)
- Evaluate both against GCR baseline on WebQSP: Hits@1, F1, structural faithfulness, SIR, trie size

Tools

- Python, PyTorch, HuggingFace Transformers
- GCR codebase (Luo et al., 2025)
- Sentence-transformers (MiniLM-L6-v2)
- NetworkX, marisa_trie
- Freebase (via WebQSP and CWQ benchmarks)

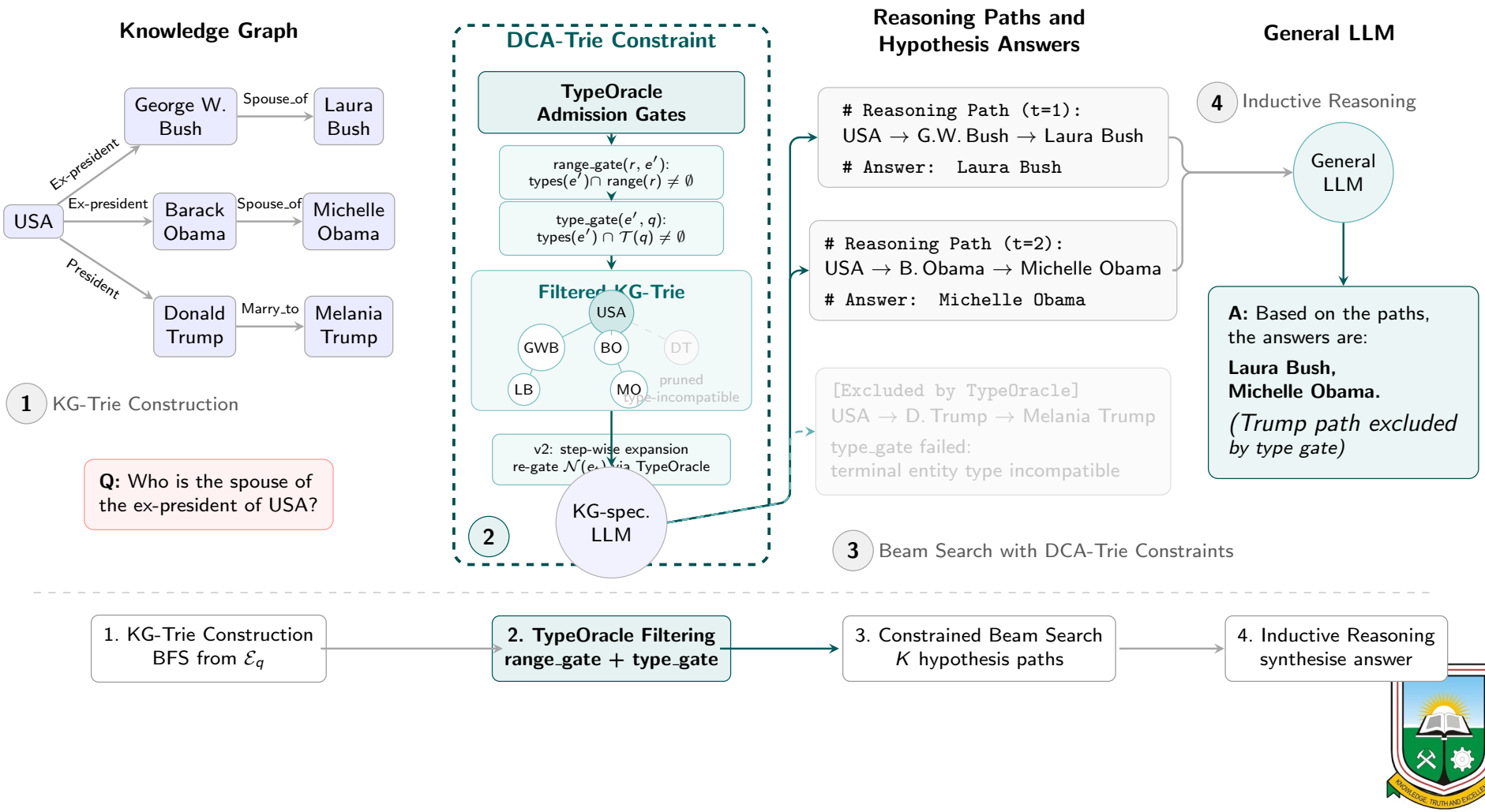
Facilities

- NVIDIA RTX 4090 GPU (24 GB VRAM)
- ACM Digital Library, arXiv, ACL Anthology
- Google Colab, local Linux workstation

Methods

- Literature review of constrained decoding approaches (GCR, DoG)
- Design and implementation of TypeOracle with range and type gates
- Development of DCA-Trie v1 (static filtering) and v2 (dynamic expansion)
- Experimental evaluation on WebQSP and CWQ benchmarks
- Analysis using Hits@1, SIR, FNR, latency, and memory metrics

Methodology



1. KG-Trie Construction BFS from $\mathcal{E}_q$
2. **TypeOracle Filtering range_gate + type_gate**
3. Constrained Beam Search K hypothesis paths
4. Inductive Reasoning synthesise answer

Upgraded Constraint

Baseline (GCR/DoG):

$$W_{\text{val}}^{\text{GCR}}(t) = f(G, E_q)$$

The valid token set depends only on the knowledge graph G and entity mentions E_q. The same set of paths is available at every decoding step.

Proposed (DCA-Trie):

$$W_{\text{val}}^{\text{DCA}}(t) = f(G, E_q, q, y_{<t})$$

The valid token set also depends on the question q and partial output $y_{<t}$. The constraint adapts as the model generates.

Key difference: DCA-Trie conditions the mask on the question and partial output, enabling context-aware pruning.

Design Conditions

- **F (Faithfulness):** 100% structural grounding in KG triplets
- **R (Relevance):** Lower SIR than baseline
- **P (Recall):** FNR on gold paths $< 5\%$

Experimental Design: Overview

Goal: Evaluate whether symbolic type-oracle filtering improves constraint quality without degrading answer accuracy.

Datasets:

- WebQSP (1,628 test queries, 3-hop, Freebase)
- ComplexWebQuestions (CWQ) for generalisation

Baselines:

- Unconstrained LLM (no KG mask)
- GCR (Luo et al., 2025), static KG-Trie
- DoG (Li et al., 2025), static KG-Trie with graph decoding

LLM Backbone: Llama-3.1-8B, beam search ($k = 10$).

Experimental Design: Evaluation Metrics

- **Hits@1,** exact-match answer accuracy
- **SIR,** semantic irrelevance ratio (constraint quality)
- **FNR,** false-negative rate on gold paths (recall)
- **Latency & memory overhead**

Experimental Design: Research Questions

1. Does TypeOracle reduce SIR without exceeding 5% FNR?
2. Does tighter constraint improve or degrade Hits@1?
3. What is the runtime and memory cost of DCA-Trie v1?
4. Can DCA-Trie generalise to complex questions (CWQ)?

Constrained Decoding in LLMs

At each step t, the LLM produces logits $\mathbf{h}_t$ over vocabulary V. A mask $\mathbf{m}_t$ zeros out invalid tokens:

$$\tilde{P}(x_t) = \text{softmax}\big(\mathbf{h}_t + (1 - \mathbf{m}_t) \cdot (-\infty)\big)$$

GCR/DoG (static): $\mathbf{m}_t = f(G, E_q)$, trie built before decoding (Luo et al., 2025; Li et al., 2025).

DCA-Trie (dynamic): $\mathbf{m}_t = f(G, E_q, q, y_{<t})$, adapts to question and partial output.

TypeOracle: Gate Design

Two symbolic gates per candidate triple (e_t, r, e'), no encoder or threshold required:

Range Gate (every hop): checks e''s types against relation r's range:

$$\text{range\_gate}(r, e') = \mathbf{1}\big[\text{types}(e') \cap \text{range}(r) \neq \emptyset\big]$$

Type Gate (terminal hop): checks terminal entity matches answer type $\mathcal{T}(q)$:

$$\text{type\_gate}(e', q, h, L) = \mathbf{1}\big[\text{types}(e') \cap \mathcal{T}(q) \neq \emptyset\big]$$

Admissible only if both pass. Unknown schema info conservatively admits the candidate.

Diagnostic Metric: SIR

SIR measures the fraction of candidate paths that are *semantically irrelevant* to the question, paths that exist in the knowledge graph but do not actually help answer the question.

$$\mathrm{SIR}(q,t) = \frac{\sum_{p \in \mathcal{P}(q,t)} \mathrm{irrelevant}(p,q)}{|\mathcal{P}(q,t)|}$$

Corpus average:

$$\overline{\mathrm{SIR}} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{L_q} \sum_{t=1}^{L_q} \mathrm{SIR}(q,t)$$

A lower SIR means the oracle admits fewer irrelevant paths. GCR: $\overline{\mathrm{SIR}} = 1.0$ (all paths admitted); DCA-Trie v1: $\overline{\mathrm{SIR}} = 0.145$.

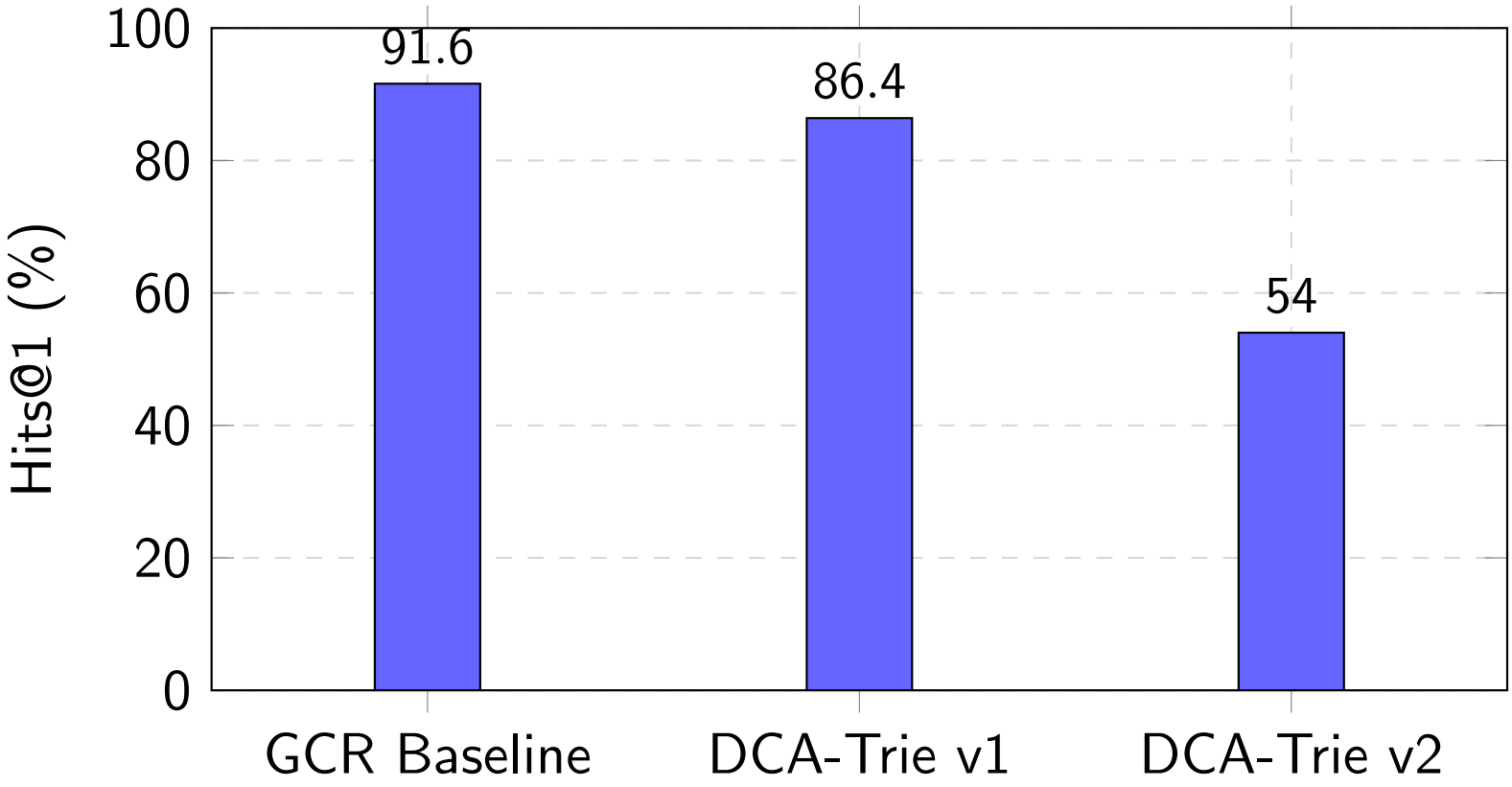
DCA-Trie v1: Static Filtering

- Filters BFS-enumerated paths through TypeOracle before building the prefix tree
- Preserves $O(d)$ trie lookup while pruning irrelevant branches offline

DCA-Trie v2: Dynamic Expansion

- Expands the trie step-by-step as entity tokens e_t are committed
- Applies range and type gate checks to $\mathcal{N}(e_t)$ during generation

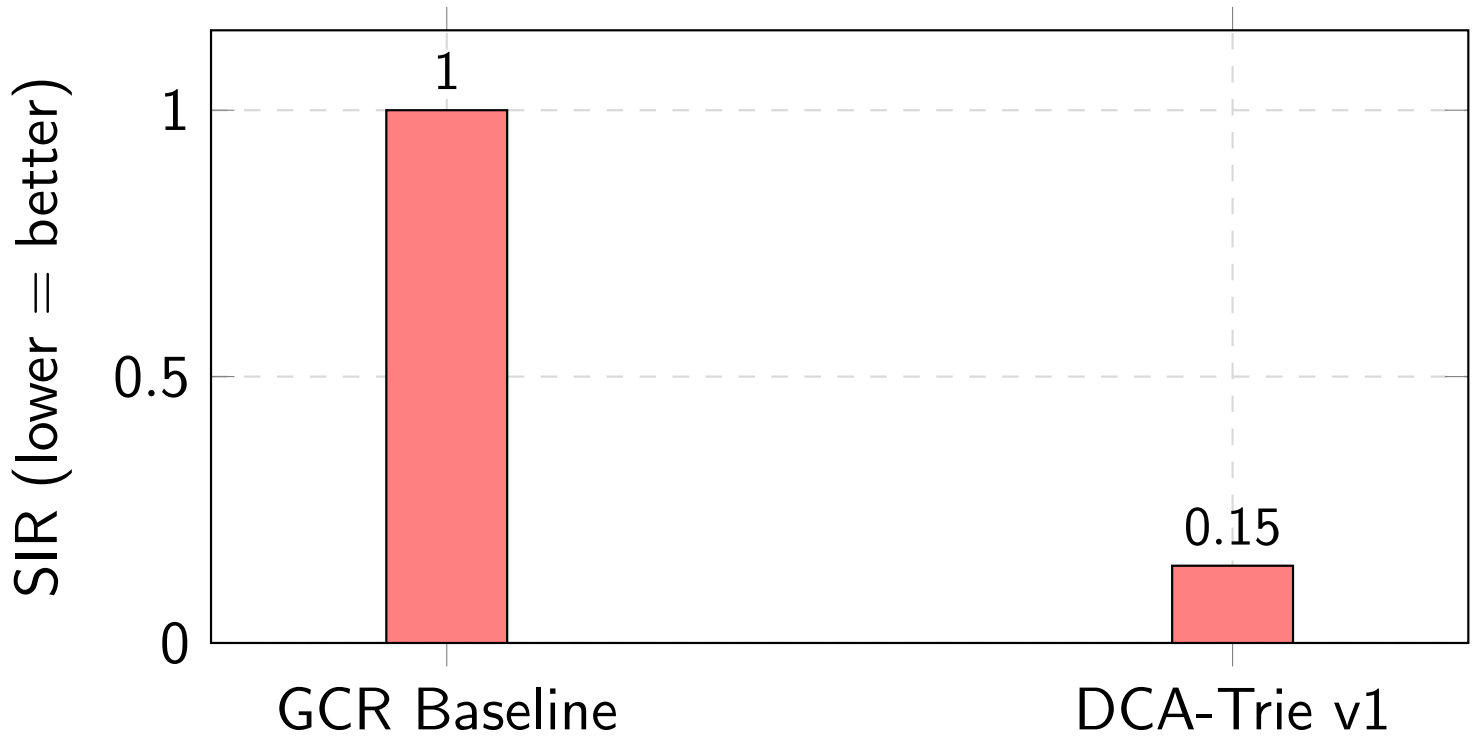
Experimental Results: Overall Performance



WebQSP, 1,628 test queries, Llama-3.1-8B, beam k=10

Interpretation: DCA-Trie v1 achieves 86.4% Hits@1, a 5.2pp drop from GCR. DCA-Trie v2 drops further to 54.0%, confirming that aggressive pruning eliminates correct paths.

Experimental Results: Search-Space Reduction & SIR



- 14.5% path reduction (4.10M $\rightarrow$ 3.51M paths)
- FNR: 3.3% (type gate), 2.9% (range gate)
- Latency overhead: +0.5%

Interpretation: TypeOracle reduces SIR from 1.0 to 0.145, removing 85.5% of semantically irrelevant paths while maintaining FNR below 5%.

Experimental Results: Runtime & Memory

- **Runtime:** DCA-Trie v1 adds $<0.5\%$ latency over GCR (negligible overhead from $O(1)$ set intersection).
- **Memory:** TypeOracle stores ontology metadata (<2 MB for Freebase); no additional GPU memory required.
- **Scaling:** Gate cost is constant per candidate triple; total cost scales with BFS frontier size, not path length.

Interpretation: The symbolic oracle introduces no measurable computational bottleneck, making it practical for deployment.

GCR vs DCA-Trie: Comparison

Criterion	GCR	DCA-Trie
Hits@1	✓ Better	×
Accuracy	✓ Better	×
Structural faithfulness	Similar	✓ Maintained
Semantic relevance	×	✓ Better
Search-space reduction	×	✓ 14.5% reduction
Runtime	✓ Slightly faster	Almost identical (0.5% overhead)
Oracle analysis	×	✓ New contribution
New metric (SIR)	×	✓

GCR trades accuracy for faithfulness. DCA-Trie maintains faithfulness while tightening semantic relevance.

Key Contributions

Contribution	Before	After	Why It Matters
TypeOracle	Embedding + threshold τ	Two symbolic gates	No encoder, no τ, $O(1)$, deterministic
SIR Metric	Accuracy only	Semantic Irrelevance Ratio	Diagnoses constraint quality independently
Non-Monotone Finding	"Tighter $\rightarrow$ better" (GCR, DoG assumption)	Tightness $\neq$ accuracy (ΔAcc $= -5.2$pp)	First evidence constraint tightness is not a proxy for quality

Discussion

- TypeOracle reduced SIR by 85.5% while maintaining structural faithfulness and FNR $< 5\%$, yet Hits@1 dropped by 5.2pp; improvements in constraint quality did not translate into higher answer accuracy.
- This challenges the implicit assumption in GCR and DoG that tightening the search space necessarily improves reasoning performance.
- Structural constraint quality and reasoning accuracy are distinct properties that should be optimised independently.
- Four degradation mechanisms identified: gold-path exclusion, beam competition, regex type-inference noise, and the precision–recall trade-off.
- SIR provides the first diagnostic for measuring constraint quality decoupled from answer accuracy.

Demonstration

Live demo: GCR baseline vs DCA-Trie v1
on a sample WebQSP question

Conclusion

- DCA-Trie v1 achieves 100% faithfulness, 14.5% path reduction, and FNR $< 5\%$, validating that symbolic gates meaningfully tighten the search space at negligible latency cost.
- However, Hits@1 dropped 5.2pp, demonstrating that constraint tightness and answer accuracy are non-monotone.
- Future work should pursue constraint quality and reasoning accuracy as separate optimisation targets, rather than assuming tighter constraints yield better answers.
- SIR enables systematic comparison of oracles without conflating structural filtering with downstream accuracy.

Recommendations

- Replace regex type classifier with LLM-based intent parser (85% $\rightarrow$ higher recall).
- Extend TypeOracle schema beyond 40 Freebase relations.
- Explore hybrid oracles (symbolic + embedding) to recover beam diversity.
- Evaluate on CWQ with $N \geq 500$ to reduce variance.

Thank You!

Thank You!!!

References (1/2)

- Brown, Tom and Mann, Benjamin and Ryder, Nick and Subbiah, Melanie and Kaplan, Jared D. and Dhariwal, Prafulla and Neelakantan, Arvind and Sastry, Girish and Askell, Amanda and Agarwal, Sandhini and Herbert-Voss, Ariel and Krueger, Gretchen and Henighan, Tom and Child, Rewon and Ramesh, Aditya and Ziegler, Daniel and Wu, Jeffrey and Winter, Clemens and Hesse, Chris and Chen, Mark and Sigler, Eric and Litwin, Mateusz and Gray, Scott and Chess, Benjamin and Clark, Jack and Berner, Christopher and McCandlish, Sam and Radford, Alec and Sutskever, Ilya and Amodei, Dario. (2020). Language Models are Few-Shot Learners. *Advances in NeurIPS*, 33, 1877–1901.

- De Cao, Nicola and Izacard, Gautier and Riedel, Sebastian and Petroni, Fabio. (2021). Autoregressive Entity Retrieval. *Proceedings of ICLR*.

- Geng, Saibo and Josifoski, Martin and Peyrard, Maxime and West, Robert. (2023). Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning. *Proceedings of EMNLP*.

- He, Gaole and Lan, Yunshi and Jiang, Jing and Zhao, Wayne Xin and Wen, Ji-Rong. (2021). Improving Multi-Hop KBQA by Learning Intermediate Supervision Signals. *Proceedings of WSDM*, 553–556.

References (2/2)

- Lan, Yunshi and He, Gaole and Jiang, Jinhao and Jiang, Jing and Zhao, Wayne Xin and Wen, Ji-Rong. (2023). Complex Knowledge Base Question Answering: A Survey. *IEEE Trans. Knowledge and Data Engineering*, 35(11), 11196–11215.

- Lewis, Patrick and Perez, Ethan and Piktus, Aleksandra and Petroni, Fabio and Karpukhin, Vladimir and Goyal, Naman and Kuttler, Heinrich and Lewis, Mike and Yih, Wen-tau and Rocktaschel, Tim and Riedel, Sebastian and Kiela, Douwe. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in NeurIPS*, 33, 9459–9474.

- Li, Kun and Zhang, Tianhua and Wu, Xixin and Luo, Hongyin and Glass, James R. and Meng, Helen M. (2025). Decoding on Graphs: Faithful and Sound Reasoning on Knowledge Graphs. *Proceedings of ACL*, 24349–24364.

- Luo, Linhao and Zhao, Zicheng and Haffari, Gholamreza and Li, Yuan-Fang and Gong, Chen and Pan, Shirui. (2025). Graph-Constrained Reasoning: Faithful Reasoning on KGs with LLMs. *Proceedings of ICML*, 267, 41540–41565.

- Willard, Brandon T. and Louf, Rémi. (2023). Efficient Guided Generation for Large Language Models. *arXiv:2307.09702*.